

# Checklist SQS Projektarbeit

Die Bewertung im Fach Software Qualitätssicherung (SQS) erfolgt anhand einer eigenständig durch die Studierenden angefertigten Projektarbeit. Die Abgabe der Projektarbeit setzt sich dabei aus folgenden Bestandteilen zusammen:

- **Gruppenarbeit:** Die Projektarbeit wird in 3er Teams durchgeführt. Jedes Teammitglied trägt gemeinschaftlich die Verantwortung für das Gesamtergebnis.
- Source-Code in einem öffentlichen Github-Repository
- Vorstellung der Projektarbeit in einem 15-minütigen Vortrag mit anschließenden 5 Minuten Fragen zur Umsetzung

## Erlaubte Tools und Sprachen

Die Projektarbeit kann in einer der folgenden Programmiersprachen geschrieben werden:

- Java
- C#
- Python
- Typescript

ChatGPT/Copilot etc. sind zugelassene Hilfsmittel. Allerdings soll es nicht der Sinn sein, sich alles von ChatGPT generieren zu lassen - das werden wir erkennen und entsprechend in die Bewertung mit einfließen lassen.

## Themenwahl

Die Themenwahl ist den Studierenden freigestellt, muss aber folgenden Komponenten enthalten:

- mindestens ein öffentlich erreichbarer Endpunkt
- mindestens ein abgesicherter Endpunkt (Login-Kontext etc.)
- mindestens 3 Schichten:
  - Frontend
  - Backend
  - Persistenzschicht (bspw. Datenbank)
- Das **Backend** muss außerdem mit mindestens einem externen Service sprechen (bspw. Google-APIs etc.).

## Anforderung an die Qualität

Darüber hinaus soll das Projekt aktuellen Softwarequalitätsstandards entsprechen:

- Die Software verfügt über ein dokumentiertes und implementiertes Testkonzept, das die komplette Testpyramide beachtet:
  - Unit-Tests
  - Integration-Tests
  - e2e Tests (mit UI-Automatisierung)
  - Penetrations Tests: Hier sind v.a. Tests gemeint, die eure abgesicherten Endpunkte testen. Das kann im Rahmen eurer Projektarbeit auch ein einfacher Integrationstest sein, der eure Sicherheitslogik testet
  - Architekturtests
  - Es gibt eine lauffähige Github-Pipeline
  - Ein Tool zur statischen Codeanalyse zeigt keine offenen Issues und eine Testabdeckung von mindestens 80%
- Es werden ausfallsichere Architekturpatterns zum Anbinden des externen Services verwendet
- Das Projekt ist nach dem arc42 Standard dokumentiert
- Wichtige Projektentscheidungen sind in ADRs festgehalten
- Die Dokumentation ist ordentlich strukturiert und öffentlich einsehbar auf readthedocs
- Das Projekt ist öffentlich zugänglich und ohne weitere Eingriffe nach Auschecken **mit maximal 2 Befehlen** lauffähig. Die Studierenden verwenden dazu entsprechende Tools (bpsw. docker compose oder ein Shellscript, das alle notwendigen Schritte ausführt)

## Vorstellung der Projektarbeit

Die Vorstellung der Projektarbeit beinhaltet folgende Punkte:

- Live-Demo der Software und der Kernfunktionalität
- Vorstellung der Analyse des Tools zur statischen Codeanalyse basierend auf dem letzten Projektcommit
- Kurzer Überblick über die Projektstruktur und die wichtigsten Komponenten (Die Studierenden verwenden dazu ein C4-Modell, das ihr mit structurizr erstellt habt)
- Vorstellung des Testkonzeptes und der verwendeten Tools
- Anschließend 5 Minuten Fragen zum Projekt durch den Dozenten oder Mitstudierende

## Notenberechnung und Peer Assessment

Die finale Individualnote setzt sich aus der **Gruppennote (NGrp)** und dem individuellen **Peer-Faktor (PF)** zusammen.

### Der Peer-Faktor

Nach Projektabschluss bewerten sich die Teammitglieder anonym gegenseitig in den Kategorien **Inhaltlicher Beitrag, Zuverlässigkeit, Kommunikation und Organisation** auf einer Skala von 1 (nicht akzeptabel) bis 5 (exzellent).

### Berechnung der Individualnote (NInd)

Die individuelle Note berechnet sich nach folgendem Bonus-Malus-Schema:

| Peer-Faktor (PF)         | Auswirkung auf Gruppennote  | Status                             |
|--------------------------|-----------------------------|------------------------------------|
| <b>PF &gt; 1.1</b>       | - 0.3 oder - 0.7 Stufen     | <b>Bonus</b> (Hohes Engagement)    |
| <b>0.9 ≤ PF ≤ 1.1</b>    | keine Änderung              | <b>Neutral</b> (Soll erfüllt)      |
| <b>0.7 ≤ PF &lt; 0.9</b> | + 0.3 oder + 0.7 Stufen     | <b>Malus</b> (Mangelnde Mitarbeit) |
| <b>PF &lt; 0.7</b>       | Festlegung auf 4.0 oder 5.0 | <b>Kritisch</b> (Kaum Beitrag)     |

### Berechnung der Gesamtnote

Um eine objektive und transparente Umrechnung der Gruppennote in die Individualnote zu gewährleisten, wird folgende Formel angewendet:

$$N_{Ind} = N_{Grp} - (PF - 1) \times S$$

Definition der Variablen:

- $N_{Ind}$ : Finale individuelle Note des Studierenden.
- $N_{Grp}$ : Vom Dozenten vergebene Note für die gesamte Projektarbeit.
- $PF$ : Der ermittelte Peer-Faktor (Individueller Punktedurchschnitt / Gruppendurchschnitt).
- $S$ : Skalierungsfaktor (Wert: 2,0). Dieser dient dazu, die Auswirkung des Peer-Feedbacks zu gewichten, damit kleine Schwankungen nicht sofort zu extremen Notensprüngen führen.

## Das Kleingedruckte

**Rundung:** Die berechnete Note wird auf die nächste gültige Notenstufe (z. B. 1,0; 1,3; 1,7 etc.) gerundet.

**Korrektur bei Unfairness:** Sollte die Git-Historie (Commits) oder die statische Codeanalyse massiv von der Peer-Bewertung abweichen, findet eine manuelle Überprüfung durch den Dozenten statt.  
+4

**Mindestleistung:** Ein Peer Assessment kann eine fachlich ungenügende Leistung nicht in eine positive Note umwandeln, wenn die Kernanforderungen (z.B. Testabdeckung < 80% oder fehlendes Testkonzept) nicht erfüllt sind.  
+2